

Oficina 2 — Do Arquivo Vazio ao LED Piscando

MPLAB X, XC8, bootloader USB e a frequência real do processador

Raoni F. S. Teixeira · Rodolfo Quadros · DENE/UFMT · 1 sessão (2 h) · sem nota · grupos de 3

OBJETIVOS DESTA SESSÃO

Ao final desta sessão o grupo deve ser capaz de:

1. Percorrer o ciclo completo editar → compilar → gravar → executar sem consultar o roteiro;
2. Explicar por que o MPLAB X, nesta bancada, apenas compila;
3. Diagnosticar as cinco falhas mais frequentes de gravação a partir do sintoma;
4. Justificar por que o `#pragma config` do seu programa não tem efeito e qual é a frequência real do processador;
5. Descrever o que é estado de repouso seguro de um atuador.

POR QUE ESTA SESSÃO NÃO TEM NOTA

Segunda das três oficinas: sem entrega e sem ponto. O objetivo é que a gravação deixe de ser um evento e vire um gesto. Um grupo que ao final da sessão consegue gravar em menos de um minuto, sem hesitar, ganhou o semestre inteiro — porque todos os roteiros seguintes gravam de dez a vinte vezes por sessão.

CONFIGURAÇÃO DE BANCADA

Chave traseira	Ligada
Cabo USB	Porta frontal (CN9), não a traseira
Chaves SWITCHS (PORTB)	Todas em OFF
CH3-3 (aquecedor)	OFF
CH3-5 (ventoinha)	ON
Demais bancos	OFF

1 Quem compila e quem grava

O manual da Exsto descreve a gravação pelo PICKit 2 embutido. **Esse caminho não funciona nesta bancada:** o firmware do kit foi regravado e a gravação passou a ser feita por um bootloader USB.

⚠ ATENÇÃO

Se você tentar gravar pelo MPLAB X, receberá PK2Error0022: PICKit 2 not found. Isso é o comportamento esperado, não defeito do equipamento.

Consequência prática: **o MPLAB X apenas compila. Quem grava é o programa do bootloader, executado separadamente.**

1.1 O que é um bootloader

O que está gravado no PIC18F4550 não é apenas a sua aplicação. No início da memória de programa existe um programa residente — o bootloader — cuja única função é receber firmware novo pela USB e escrevê-lo no restante da memória.

O ponto que mais causa confusão é o tempo:

⚠️ ATENÇÃO

O bootloader só aceita conexão durante os primeiros **4 a 5 segundos** após o kit ser ligado ou reiniciado. Passada essa janela, ele entrega o controle à aplicação e deixa de responder.

O programa no PC continua aberto e **não exibe mensagem de erro**. O botão Connect simplesmente não encontra nada.

Cinco segundos é tempo de sobra para clicar sem pressa — desde que o programa já esteja aberto e a janela visível. O erro típico não é lentidão no clique: é reiniciar o kit **antes** de abrir o programa, ou procurar o botão Connect depois do reset.

A sequência correta, portanto, é:

1. Abrir o programa do bootloader, deixando Connect visível na tela;
2. Pressionar o botão de reset (SW9);
3. Clicar em Connect nos 4 a 5 segundos seguintes;
4. Se não conectar, repetir — reset e Connect logo em seguida.

2 Parte 1 — Criar e compilar

1. MPLAB X → **File** → **New Project**
2. **Microchip Embedded** → **Application Project(s)**
3. Device: PIC18F4550
4. Tool: **No Tool**
5. Compiler: XC8
6. Nomeie o projeto e conclua.

OBSERVAÇÃO

Se o MPLAB X pedir para baixar o **Device Family Pack** do PIC18F4550, autorize. Sem ele o compilador não encontra o cabeçalho do dispositivo e a compilação falha logo na primeira linha.

A escolha **No Tool** não é um contorno improvisado: ela declara ao ambiente que não há gravador conectado, o que é exatamente a verdade nesta bancada.

Adicione o arquivo-fonte fornecido pelo professor em **Source Files** → **Add Existing Item**, e compile com o martelo da barra de ferramentas (**Clean and Build**, F11).

⚠️ ATENÇÃO

Não use o botão de Debug. O build de depuração gera .elf e não gera .hex — e sem .hex não há o que gravar. Este é o erro mais frequente da sessão.

O arquivo deve aparecer em:

```
<projeto>/dist/default/production/<nome>.production.hex
```

Se existir apenas a pasta debug, você compilou no modo errado.

TAREFA

Anote o caminho completo do arquivo gerado. Depois abra o .hex em um editor de texto e olhe as primeiras linhas. Não é preciso entender o formato — é preciso saber que ele é texto, que tem endereço e dado em cada linha, e que é isso, e não o seu arquivo .c, que atravessa o cabo USB.

3 Parte 2 — Gravar

1. Cabo USB na porta frontal (CN9);
2. Abrir o programa **Bootloader PIC18 — XM118** (USB HID Bootloader);

3. Pressionar SW9 (reset);
4. Connect dentro de 4 a 5 segundos;
5. **Buscar arq HEX** → selecionar o `.production.hex` da Parte 1;
6. **Begin uploading**;
7. Ao final, pressionar SW9 novamente para executar o firmware novo.

A janela de histórico deve mostrar a sequência de apagamento, escrita, conclusão e desconexão, terminando com o aviso de que é preciso reiniciar o dispositivo para reentrar em modo bootloader.

OBSERVAÇÃO

Repare que a última mensagem é do próprio programa e confirma o que foi explicado acima: para gravar de novo, é preciso reiniciar de novo. O equipamento está lhe dizendo como usá-lo — ler a mensagem é parte do ofício.

TAREFA

Treino cronometrado. Grave o mesmo arquivo três vezes seguidas. Na terceira, meça quanto tempo levou do reset ao LED piscando. Se passou de um minuto, grave uma quarta vez.

Não é exercício de velocidade: é para que a mecânica saia da cabeça e libere atenção para o problema real nas sessões seguintes.

4 Catálogo de falhas por sintoma

Este é o conteúdo mais útil da oficina. Ele está organizado por **sintoma** — que é o que você observa — e não por causa, que é o que você quer descobrir.

Sintoma	Causa provável	Ação
Não existe pasta <code>production</code>	Compilou pelo botão de Debug	Recompilar com o martelo
PK2Error0022	Tentou gravar pelo MPLAB	Usar o programa do bootloader
Bootloader não conecta	Janela de 4 a 5 s expirou	Reset e Connect em seguida
Não conecta, persistente	MPLAB X aberto, ou cabo na USB traseira	Fechar o MPLAB; trocar de porta
Gravou, nada acontece	Faltou o RESET depois do upload	Pressionar SW9
Comportamento antigo persiste	O <code>.hex</code> gravado não é o que você compilou	Conferir data e caminho do arquivo
Compila, grava, mas o tempo está errado	Frequência de operação diferente da suposta	Ver a seção seguinte

Tabela 1: Diagnóstico rápido do ciclo de gravação.

CONCEITO CENTRAL

Repare na estrutura da tabela. Ela é o modelo de raciocínio de toda a disciplina: parte-se do que se **observa**, levanta-se a causa mais provável e age-se para confirmá-la ou descartá-la. Depurar não é ler o código até a iluminação chegar — é reduzir o espaço de hipóteses com medidas.

5 Quem manda no clock

Esta seção existe porque o erro que ela previne é o mais caro do semestre.

O PIC18F4550 tem bits de configuração — entre eles os que selecionam o oscilador e os divisores — que são gravados junto com o firmware e escolhem a frequência de operação. Em C, eles são escritos com diretivas `#pragma config`.

⚠️ ATENÇÃO

Nesta bancada, o `#pragma config` do seu programa é silenciosamente ignorado. Quem grava é o bootloader, e o bootloader é dono dos bits de configuração: ele não os sobrescreve com os do seu arquivo.

O compilador não avisa. O gravador não avisa. O programa roda. Ele apenas roda em uma frequência diferente da que você escreveu.

O bootloader configura a cadeia de clock para atender ao requisito da USB, mas a frequência que chega ao núcleo do processador — a que governa `__delay_ms`, a contagem dos temporizadores e a taxa da serial — é de **16 MHz**.

CONCEITO CENTRAL

Toda derivação de tempo desta disciplina parte de 16 MHz. Preload de temporizador, período de PWM, divisor de baud rate: todos.

A constante do compilador precisa refletir isso:

```
#define _XTAL_FREQ 16000000UL
```

Se ela estiver errada, `__delay_ms(500)` não produz meio segundo, e o erro é proporcional — não aparece como falha, aparece como um sistema que funciona com o tempo errado. Que é muito pior, porque não chama atenção.

🔧 EXPERIMENTE E ANOTE

A medida que fecha a questão. Grave um programa que alterne um pino a cada 300 ms usando `__delay_ms`, com `_XTAL_FREQ` declarado como 48 MHz.

1. Meça o período no osciloscópio, com a garra no GND — como na Oficina 1.
2. Compare o valor medido com os 300 ms pretendidos.
3. Calcule a razão entre medido e pretendido.
4. A partir dessa razão e do valor declarado, deduza a frequência real.

Anote a conta. Ela é a única evidência que convence: não se está pedindo que você acredite no professor, e sim que meça.

OBSERVAÇÃO

Vale registrar por que o valor de 48 MHz aparece em tanta documentação e em tanto código encontrado na internet. Ele é real — é a frequência exigida pelo periférico USB — e é o que muitos projetos de PIC18F4550 configuram. O que não é verdade é que ele valha automaticamente para o núcleo nesta configuração de bootloader.

Código copiado da internet traz junto as premissas de quem o escreveu. Esta é a primeira delas que você vai ter que desmontar neste semestre; não será a última.

6 Estado de repouso seguro

Volte à sua anotação da Oficina 1: a ventoinha ligava sozinha com o kit recém energizado. Agora vale explicar.

Um pino do microcontrolador tem duas propriedades independentes: a **direção** (entrada ou saída, registrador TRIS) e o **valor** que ele impõe quando é saída (registrador LAT). Ao ligar, todos os pinos são entradas, e o que o atuador faz depende apenas do circuito externo.

O instante crítico é a transição:

```
LATC  &= ~0b00000110;  /* linha A: define o valor de repouso */
TRISC &= ~0b00000110;  /* linha B: agora o pino passa a ser saída */
```

EXPERIMENTE E ANOTE

Localize essas duas linhas no código fornecido e responda, **antes** de testar: o que aconteceria se A e B fossem trocadas de ordem?

Pense no que o pino impõe no instante em que deixa de ser entrada e passa a ser saída, se ninguém tiver definido antes o valor que ele deve impor.

Depois teste, observando a ventoinha.

CONCEITO CENTRAL

Escrever no registrador de dados **antes** de configurar a direção é a diferença entre um atuador que acorda desligado e um que acorda em estado indefinido — tipicamente um pulso curto, mas real.

Com uma ventoinha, o pulso é inofensivo e didático. Com a resistência de aquecimento seria desagradável. Em um sistema de potência real — um disjuntor, uma chave de manobra, o disparo de um tiristor — seria um acidente. A ordem dessas duas linhas é engenharia de segurança escrita em duas instruções.

7 Fechamento

TAREFA

Devolva a bancada ao estado da Oficina 1: chaves em OFF, kit desligado na ordem correta, cabos organizados.

TAREFA

Para a Oficina 3. Responda por escrito, sem compilar:

1. Qual a diferença entre $LATD = 0x01$ e $LATD |= 0x01$?
2. Se LATD vale 0b10110000 e você executa $LATD |= 0b00000001$, quanto passa a valer?
3. E se executasse $LATD = 0b00000001$?

Se não souber, escreva o que você **acha** que acontece em cada caso, e por quê.